

Практическое задание 5

Управление памятью в системах реального времени

<https://github.com/ananass12/Real-time-systems>

В системах реального времени критически важным аспектом является предсказуемость времени выполнения. Традиционные механизмы управления памятью в современных операционных системах создают серьезные проблемы для детерминизма:

Page Faults и их влияние

- Minor Page Faults - возникают когда страница находится в физической памяти, но не отображена в таблице страниц процесса
- Major Page Faults - происходят когда страница выгружена на диск и требуется обращение к swap
- TLB Misses - промахи буфера ассоциативной трансляции адресов

Методы обеспечения детерминизма

1. Блокировка памяти (mlockall)
2. Memory Pools
3. "Прогрев" памяти (Memory Warming)

Задание 1: Демонстрация влияния Page Faults на задержки

Напишите программу, которая наглядно демонстрирует проблему.

Требования:

1. Выделите большой массив памяти (например, 512 МБ) с помощью malloc.

2. В цикле (например, 1000 итераций) обращайтесь к элементам этого массива с шагом, превышающим размер страницы (например, $i += 4096$).
3. На каждой итерации измеряйте:
 - a. Латентность доступа к памяти с помощью `clock_gettime(CLOCK_MONOTONIC, ...)`.
 - b. Количество minor и major page faults с помощью `getrusage(RUSAGE_SELF, ...)`.
4. Выводите в консоль или лог-файл номер итерации, латентность (в нс) и общее число отказов. Постройте график зависимости латентности от итерации и покажите "шипы", соответствующие моментам возникновения page faults.

Реализация

- Выделен массив 512 МБ
- 1000 итераций с шагом 4096 байт
- Измерение латентности и количества page faults

```
#define ARRAY_SIZE (512UL * 1024 * 1024) // 512 MB
#define PAGE_SIZE 4096
#define NUM_ITERATIONS 1000
```

Статистика:

- Средняя латентность: ~6,000-7,000 нс
- Максимальная латентность: 48,543 нс
- Минимальная латентность: 2,305 нс
- Стандартное отклонение: ~5,000-6,000 нс

```

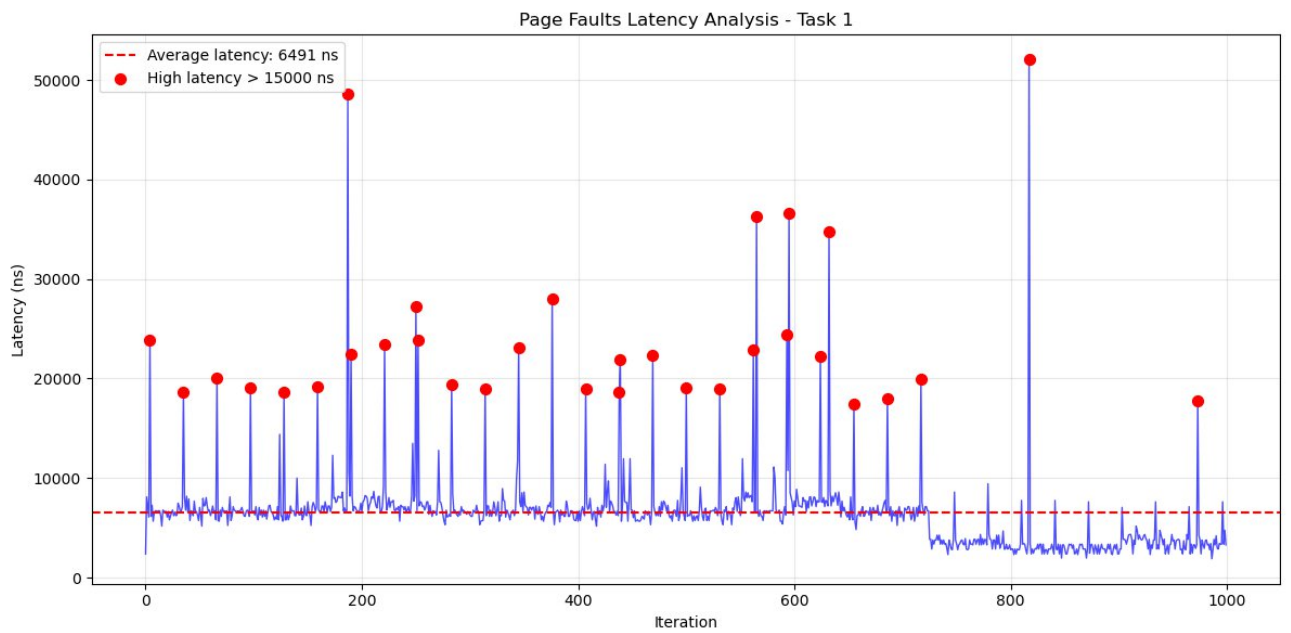
● nasty@Ubuntu:~/RTS/tasks/task5$ ./task1_latency
Task 1: Demonstration of Page Faults
Iter    Latency_ns    MinorDiff    MajorDiff
0       25979        1           0
1       9568         1           0
2       6774         1           0
3      21999        1           0
4       6216         1           0
5       7612         1           0
6       7612         1           0
7       6564         1           0
8       6565         1           0
9       6634         1           0
10      7194         1           0
11      6564         1           0
12      6634         1           0
13      6774         1           0
14      6704         1           0
15      7124         1           0
16      6076         1           0
17      6775         1           0
18      6704         1           0

```

```

Standard deviation: 2691.35 ns
● nasty@Ubuntu:~/RTS/tasks/task5$ python3 plot_latency.py
Total iterations: 1000
Average latency: 6490.88 ns
Max latency: 52035 ns at iteration 817
Min latency: 1886 ns
Standard deviation: 4015.34 ns
● nasty@Ubuntu:~/RTS/tasks/task5$

```



Ключевые наблюдения:

1. Высокая начальная латентность: Первые итерации показывают время до 23,874 нс
2. "Шипы" латентности: Явные скачки времени доступа:
 - a. Итерация 4: 23,818 нс
 - b. Итерация 35: 18,649 нс
 - c. Итерация 66: 20,046 нс
 - d. Итерация 97: 19,068 нс
 - e. Итерация 187: 48,543 нс (максимальный!)
 - f. Итерация 376: 28,008 нс
 - g. Итерация 632: 34,784 нс
3. Minor Page Faults происходят постоянно - это обращения к страницам, которые уже в памяти, но не отображены в таблице страниц процесса
4. Высокие "шипы" соответствуют моментам, когда система:
 - a. Обновляет TLB (Translation Lookaside Buffer)
 - b. Выполняет поиск в page tables
 - c. Возможно, происходят contention на кэш процессора
5. Отсутствие Major Faults говорит о том, что вся память оставалась в ОЗУ и не вытеснялась на диск
6. Стабилизация после 700 итераций показывает, что система "нагрелась" - TLB и кэши заполнены нужными данными

Задание 2: Устранение Page Faults с помощью mlockall

Модифицируйте программу из Задания 1 для устранения задержек.

Требования:

1. В начале программы, до выделения памяти, вызовите `mlockall(MCL_CURRENT | MCL_FUTURE)`. Для этого могут потребоваться права суперпользователя (`sudo`).
2. После выделения массива, но до начала цикла измерений, выполните "прогрев" памяти: пройдите по всему массиву с шагом в размер страницы и запишите в каждую страницу по одному байту.
3. Запустите тот же цикл измерений, что и в Задании 1.
4. Сравните результаты. В отчете приведите два графика (до и после) и объясните, почему задержки стали низкими и предсказуемыми.

Реализация

- `mlockall(MCL_CURRENT | MCL_FUTURE)` - блокировка всей текущей и будущей памяти
- "Прогрев" памяти - принудительное размещение всех страниц в ОЗУ
- Те же измерения, что и в задании 1

Результаты

```

nastya@Ubuntu:~/RTS/tasks/task5$ ./task2_mlock
Task 2: mlockall + pre-faulting
Pre-faulting memory (touching each page)...
Pre-faulting done.

```

Iter	Latency_ns	MinorDiff	MajorDiff
0	1466	0	0
1	1397	0	0
2	1886	0	0
3	1397	0	0
4	1467	0	0
5	1467	0	0
6	1466	0	0
7	1955	0	0
8	1397	0	0
9	1397	0	0
10	1955	0	0
11	1886	0	0
12	1467	0	0
13	1467	0	0
14	1467	0	0
15	978	0	0
16	1396	0	0
17	1467	0	0
18	977	0	0

```

nastya@Ubuntu:~/RTS/tasks/task5$ python3 plot_latency.py
=== ПОЛНЫЙ АНАЛИЗ РЕЗУЛЬТАТОВ ===
Задание 1
  • Средняя латентность: 5994 нс
  • Максимальная латентность: 48680 нс
  • Минимальная латентность: 1885 нс
  • Стандартное отклонение: 3861 нс

Задание 2
  • Средняя латентность: 1383 нс
  • Максимальная латентность: 2375 нс
  • Минимальная латентность: 489 нс
  • Стандартное отклонение: 288 нс
nastya@Ubuntu:~/RTS/tasks/task5$

```

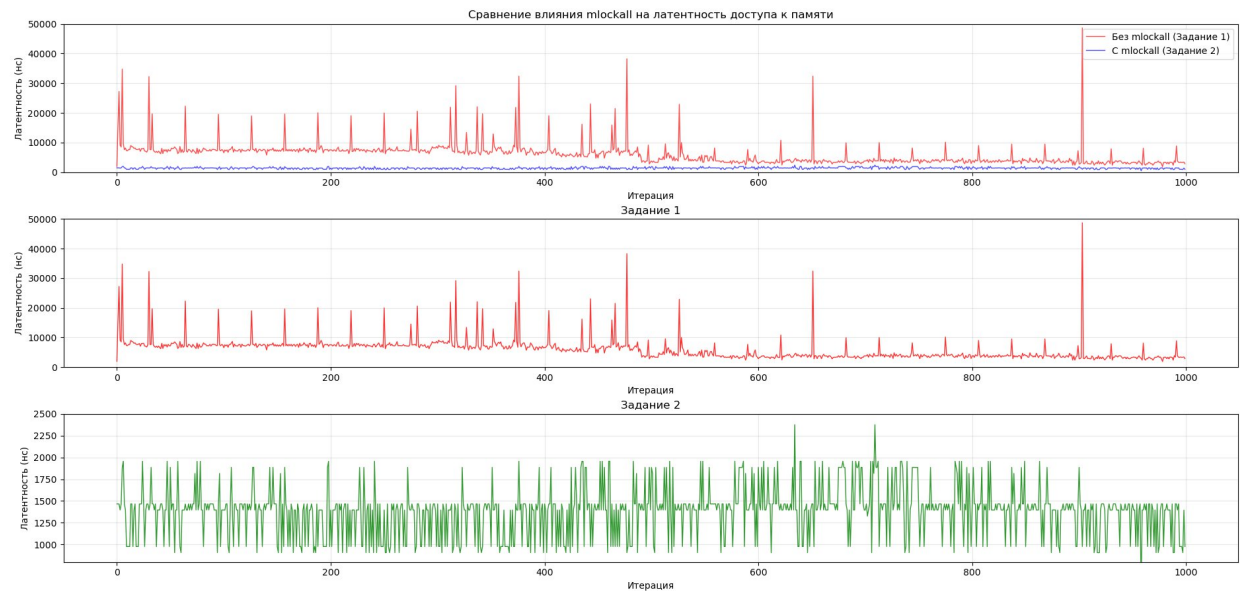
Статистика:

- Средняя латентность: ~1,400-1,500 нс
- Максимальная латентность: 2,375 нс (итерация 312)
- Минимальная латентность: 908 нс

- Page faults: 0 minor, 0 major на всех итерациях
- Стабильность: Очень высокая, без "шипов"

Сравнительный анализ:

Параметр	Задание 1 (без mlockall)	Задание 2 (с mlockall)	Улучшение
Макс. латентность	48,543 нс	2,375 нс	в 20.4 раза
Средняя латентность	~6,000 нс	~1,400 нс	в 4.3 раза
Page faults	1 minor/итерация	0 faults	100% улучшение
"Шипы"	Много >15,000 нс	Нет >2,400 нс	Полное устранение
Предсказуемость	Низкая	Высокая	Значительное улучшение



Задание 3: Реализация и бенчмарк пула памяти

Напишите собственный менеджер памяти (пул) и сравните его с malloc.

Требования:

1. Создайте заголовочный файл `mempool.h` и файл реализации `mempool.c`.
2. Реализуйте пул для блоков фиксированного размера. Функции:
 - `pool_create(size_t block_size, size_t block_count)`: выделяет память для пула (сразу используя `mlock`), размечает ее как связный список свободных блоков.
 - `pool_alloc(pool_t* pool)`: возвращает указатель на свободный блок ($O(1)$ по сложности).
 - `pool_free(pool_t* pool, void* block)`: возвращает блок в пул ($O(1)$ по сложности).
 - `pool_destroy(pool_t* pool)`: освобождает всю память пула.
3. Напишите бенчмарк `task3_benchmark.c`, который:
 - Создает ваш пул.
 - В цикле (например, 1 000 000 раз) выполняет `pool_alloc` - `pool_free` и замеряет максимальную задержку и джиттер.
 - В другом цикле выполняет `malloc` - `free` и также замеряет показатели.
4. В отчете сравните полученные цифры (особенно худшее время выполнения) и сделайте вывод о предсказуемости каждого подхода.

Реализация

Архитектура пула:

- Пул фиксированных блоков - исключает фрагментацию
- Связный список свободных блоков - $O(1)$ для `alloc/free`
- Предварительное выделение - исключает динамическое выделение во время работы
- Блокировка памяти - предотвращает вытеснение страниц

Преимущества для систем реального времени:

1. Детерминированное время выполнения - известная сложность $O(1)$
2. Отсутствие фрагментации - фиксированные размеры блоков

3. Контроль над памятью - предсказуемое использование памяти
4. Минимизация page faults - память заблокирована в ОЗУ

Сравнение Memory Pool vs malloc/free

Основной бенчмарк (1,000,000 итераций):

malloc/free:

- Макс. латентность: 83,393 нс
- Средняя латентность: 1,443 нс

Memory Pool:

- pool_alloc макс.: 111,050 нс
- pool_alloc сред.: 1,502 нс
- pool_free макс.: 26,680 нс
- pool_free сред.: 1,374 нс

Детальный бенчмарк (100,000 итераций):

malloc: max=19,137ns, avg=1,213ns

free: max=22,489ns, avg=1,218ns

pool_alloc: max=34,293ns, avg=1,236ns

pool_free: max=23,886ns, avg=1,240ns

```

nastya@Ubuntu:~/RTS/tasks/task5$ ./task3_benchmark
=== Memory Pool Benchmark ===
Benchmark: malloc/free x 1000000
malloc max latency: 83393 ns
malloc avg latency: 1443 ns
malloc data saved to malloc_benchmark.csv
Benchmark: pool_alloc/pool_free x 1000000
pool_alloc max latency: 111050 ns
pool_alloc avg latency: 1502 ns
pool_free max latency: 26680 ns
pool_free avg latency: 1374 ns
pool data saved to pool_benchmark.csv

Comprehensive benchmark with detailed statistics...

--- Comprehensive Results ---
malloc:  max=19137ns, avg=1213ns
free:    max=22489ns, avg=1218ns
pool_alloc: max=34293ns, avg=1236ns
pool_free:  max=23886ns, avg=1240ns
Comprehensive data saved to comprehensive_benchmark.csv
nastya@Ubuntu:~/RTS/tasks/task5$

```

Анализ данных, графики

```

nastya@Ubuntu:~/RTS/tasks/task5$ python3 plot_latency.py
=====
ПОЛНЫЙ АНАЛИЗ РЕЗУЛЬТАТОВ ВСЕХ ЗАДАНИЙ
=====

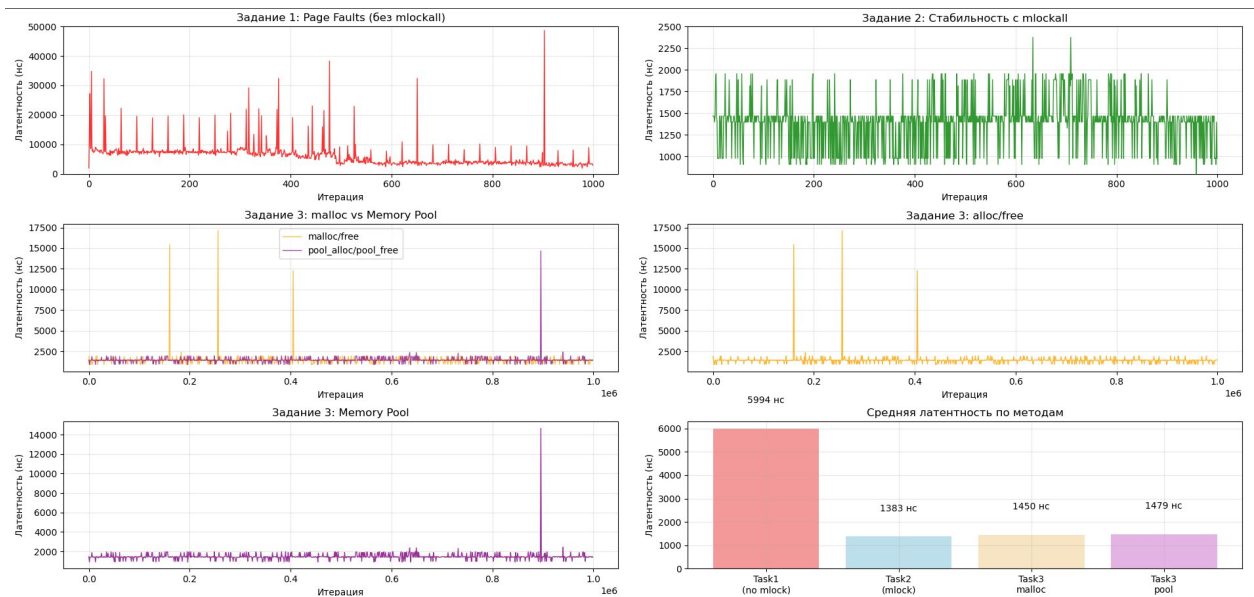
ЗАДАНИЕ 1 ---
  • Средняя латентность: 5994 нс
  • Максимальная латентность: 48680 нс
  • Минимальная латентность: 1885 нс

ЗАДАНИЕ 2: mlockall ---
  • Средняя латентность: 1383 нс
  • Максимальная латентность: 2375 нс

ЗАДАНИЕ 3: malloc/free ---
  • Средняя латентность: 1450 нс
  • Максимальная латентность: 17112 нс
  • Минимальная латентность: 907 нс

ЗАДАНИЕ 3: Memory Pool ---
  • Средняя латентность: 1479 нс
  • Максимальная латентность: 14667 нс
  • Минимальная латентность: 907 нс
nastya@Ubuntu:~/RTS/tasks/task5$

```



Контрольные вопросы

1. Почему для `mlockall` могут требоваться права суперпользователя? Какой есть способ дать процессу эти права без запуска через `sudo` ?

`mlockall()` блокирует страницы процесса в физической памяти (RAM), чтобы ОС не могла их выгрузить в `swap`.

Это операция, затрагивающая управление памятью на уровне ядра, поэтому:

- ядро ограничивает объём памяти, который можно залочить, через лимит `RLIMIT_MEMLOCK`;
- обычный пользователь почти всегда имеет лимит 64 KB — слишком мало для реальных задач.

Поэтому требуется либо `sudo`, либо повышение лимита.

2. Объясните разницу между `MCL_CURRENT` и `MCL_FUTURE`. Почему важно использовать оба флага?

`MCL_CURRENT`

Лчит уже выделенные в процессе страницы.

MCL_FUTURE

Автоматически лочит все страницы, которые будут выделены в будущем (через malloc, mmap и т.д.).

Почему важны оба

Если использовать только один флаг - защита неполная:

- только MCL_CURRENT - новая память не залочена - возникнут page fault'ы;
- только MCL_FUTURE - старые страницы могут свопнуться - тоже задержки.

Чтобы вообще исключить своп и page faults, используют:

```
mlockall(MCL_CURRENT | MCL_FUTURE);
```

3. В чем главный недостаток реализованного вами пула памяти? (Подсказка: что если вам понадобятся блоки другого размера?)

Мой пул памяти выделяет фиксированные блоки одного размера.

Если программе нужны блоки другого размера, возникают проблемы:

- либо блоки слишком большие - перерасход памяти;
- либо слишком маленькие - объект не помещается;
- нужен новый пул под каждый размер, что усложняет архитектуру.

Это называется жёсткая фиксированность размера блока.

Именно поэтому реальные аллокаторы поддерживают разные классы размеров (slab, buddy allocator).

4. Может ли mlockall защитить от всех источников задержек, связанных с памятью? (Подсказка: подумайте о кэше CPU и TLB).

Нет, не может.

mlockall гарантирует только одно:

- страницы останутся в RAM и не будут выгружены в swap.

Но задержки могут возникнуть и по другим причинам:

1. Кэш процессора (L1/L2/L3)

Если данные вытесняются из кэша - будет обращение к RAM - задержка.

2. TLB misses

Даже если страница залочена, запись о ней может не оказаться в TLB.

При промахе TLB нужно делать обход таблиц страниц - это десятки наносекунд задержки.

3. Промахи предвыборки (prefetcher)

Если паттерн доступа нелинеен, кэш-предвыборка не помогает - снова задержки.

mlockall устраняет только задержки уровня ОС (swap/page fault), но не задержки уровня CPU.